



COSC3118 – Artificial Intelligence

Assignment 3: Reinforcement Learning

Nguyen Quoc Trong Nghia (s3343711)

RMIT university

Master of Artificial Intelligence

School of Science, Engineering & Technology

Task 2

Question 1:

To verify my value iteration and Q-Learning agents were acting as expected, I used both manual testing and debug logging during training. For example, in the screenshot, I ran `gridworld.py` in manual mode (param: `-m`) on `MazeGrid` layout. This allowed me to observe step by step transition.

```
reinforcement + (main) 17:14 python gridworld.py -r 0.9 -n 0.2 -e 0.3 -l -.5 -i 10 -k 1 -m -g MazeGrid
## Disabling Agents in Manual Mode (-m) ##

RUNNING 1 EPISODES

BEGINNING EPISODE: 1

Started in state: (0, 0)
Took action: east
Ended in state: (1, 0)
Got reward: 0.9

Started in state: (1, 0)
Took action: east
Ended in state: (2, 0)
Got reward: 0.9

Started in state: (2, 0)
Took action: east
Ended in state: (3, 0)
Got reward: 0.9

Started in state: (3, 0)
Took action: north
Ended in state: (3, 1)
Got reward: 0.9

Started in state: (3, 1)
Took action: north
Ended in state: (3, 2)
Got reward: 0.9

Started in state: (3, 2)
Took action: west
Ended in state: (2, 2)
Got reward: 0.9

Started in state: (2, 2)
Took action: north
Ended in state: (2, 2)
Got reward: 0.9

Started in state: (2, 2)
Took action: north
Ended in state: (2, 3)
Got reward: 0.9

Started in state: (2, 3)
Took action: north
Ended in state: (2, 4)
Got reward: 0.9

Started in state: (2, 4)
Took action: east
Ended in state: (3, 4)
Got reward: 0.9

Started in state: (3, 4)
Took action: exit
Ended in state: TERMINAL_STATE
Got reward: 1

EPISODE 1 COMPLETE: RETURN WAS 6.210572479200001

AVERAGE RETURNS FROM START STATE: 6.210572479200001
```

All available params are displayed with this command: `python gridworld.py -h`.

By manual walking through an entire episode, I could confirm that the agent's rewards were being calculated. Then I adjusted the params(`-r`, `-n`, `-e`, `-l`, `-i`, `-k`) to see the differences in value with the same number of manual moves.

Once the implementation is done, I verified against other layouts to see the results. In smaller grids, it was easy to reason about the optimal paths manually and check whether my learned policy matched expectation. In larger layouts, I relied more on checking performance metrics and watching the agent's gameplay to ensure it was running correctly.

I did same approach for all other questions with manual testing with console logs.

Question 2:

One of the courses I studied was COSC3011 Programming Autonomous Robots studying AWS DeepRacer for reinforcement learning, they used lots of parameters in order to shape the rewards. For example, they used these parameters: distance from center line, speed, steering angle, progression, time penalties.

Similarly, in the Pacman world, I would suggest instead of just rewarding Pacman at terminal states(win/lose), we can shape the reward with multiple parameters. For example:

- Distance to nearest food and pellets: encourage progression
- Distance to nearest ghost: penalize risky states and encourage safety first
- Power pellet usage: bonus for eating power pellet, including extra for eating ghosts to have more points
- Time based performance: the less time to eat all foods and pellets, the higher reward receives.

Task 3

For this assignment, I used free version of [OpenAI's ChatGPT](#) as an assisted learning tool. The main purpose was to clarify reinforcement learning concepts and support the implementation of Value Iteration, QLearning and ApproximateQLearning agents for Pacman project.

At the start, I asked the ChatGPT to explain the definition of MDP, Value Iteration, Policies, Q-Learning algorithm, Epsilon Greedy and Approximate Q-Learning algorithm at conceptual level. It provided an overview of how Q-learning works in term of reinforcement learning, including the role of state(s), action(a), rewards, learning rate(α), discount(γ)...etc. It gave me a stronger foundation of understanding the temporal differences error update the rule behind the algorithm.

Then, I asked ChatGPT to understand the reinforcement learning codebase. To me, the codebase is somehow nontrivial to understand at glance. So ChatGPT helped me understand the functions that I needed to implement.

For example, the question is something like this screenshot below

Q-Learning Agent

Functions you should fill in:

- computeValueFromQValues
- computeActionFromQValues
- getQValue
- getAction
- update

Instance variables you have access to

- self.epsilon (exploration prob)
- self.alpha (learning rate)
- self.discount (discount rate)

Functions you should use

- self.getLegalActions(state)

which returns legal actions for a state

explain me the self.epsilon, alpha and discount in this context?



Great question 🍌 — those three (`epsilon`, `alpha`, `discount`) are the **hyperparameters** that control how your **Q-Learning Agent** learns. Let's unpack them in the context of CS188 Project 3:

◆ 1. `self.epsilon` (exploration probability)

- This controls the **exploration vs exploitation tradeoff**.
- With probability ϵ , the agent chooses a **random action** (exploration).
- With probability $1 - \epsilon$, it chooses the **best action so far** (exploitation, i.e. `argmaxa Q(s,a)`).
- Why?
 - If $\epsilon = 0$ → the agent is greedy (may get stuck in a suboptimal path).
 - If $\epsilon = 1$ → the agent moves randomly (never settles into an optimal policy).
 - Typically, ϵ is small (e.g., 0.05 or 0.1) so the agent explores occasionally but mostly exploits.

◆ 2. `self.alpha` (learning rate)

- This controls how much new information overrides old information.

Unlock more with Plus
ChatGPT Plus gives you higher limits, smarter models, and Sora for video.

Upgrade X

After understanding the algorithm, I implemented the algorithm in python, with mappings of both algorithm context and python context.

```
class QLearningAgent(ReinforcementAgent): 6 usages (2 dynamic)
    def update(self, state, action, nextState, reward: float):
        """ YOUR CODE HERE """
        #  $Q(s,a) \leftarrow Q(s,a) + \alpha * [R(s,a,s') + \text{discount} * \max_{a'} Q(s',a') - Q(s,a)]$ 
        # where
        # s(state) = current state = state
        # a(action) = action taken = action
        # s' = next state = nextState
        # R(s, a, a') = reward received = reward
        # alpha = learning rate = self.alpha
        # discount = self.discount
        #  $[R(s,a,s') + \text{discount} * \max_{a'} Q(s',a') - Q(s,a)]$  = temporal difference error = temporal_difference_error

        old_q_value = self.getQValue(state, action) # before update
        max_q_value = self.computeValueFromQValues(nextState)

        temporal_difference_error = reward + self.discount * max_q_value - old_q_value
        new_q_value = old_q_value + self.alpha * temporal_difference_error

        self.values[(state, action)] = new_q_value

        # self.debug_update(state, action, reward, old_q_value, max_q_value, temporal_difference_error, {})

    def getPolicy(self, state): 2 usages (2 dynamic)
        return self.computeActionFromQValues(state)
```